

rate exceeds predefined thresholds (for example, a failure rate threshold of 50), appropriate protective actions are triggered.

Under normal operation, when no failures or slow calls are detected, the circuit breaker remains in the CLOSED state, allowing uninterrupted service. By carefully monitoring metrics such as failure rates, buffered calls, and slow calls, the circuit breaker helps maintain system resilience while minimising performance degradation. This approach contributes significantly to fault tolerance by preventing systemwide failures and ensuring high availability and reliability, even during adverse conditions.

The goal of fault tolerance, supported by mechanisms like circuit breaking, is to minimise the impact of failures, ensuring continuous, reliable service for mission-critical applications.

Circuit breaker pattern

The circuit breaker pattern is a design pattern, represented in Figure 8, used to detect failures in a system and prevent them from cascading. In a microservices architecture or distributed system, services often depend on other services to function properly. When one service becomes unavailable or experiences issues, it can cause a chain reaction of failures across the system. The circuit breaker pattern helps mitigate this by monitoring service calls and breaking the circuit (that is, preventing further calls) when a threshold of failures is reached. Listing 1 illustrates the implementation of a circuit breaker using RestTemplate.

After a certain time or recovery period, the circuit breaker allows a limited number of test requests to check if the service has recovered. If the service is healthy again, the circuit breaker closes, and normal operation resumes. This approach helps improve system resilience, reduce downtime, and ensure that failures in one service do not impact the entire system.

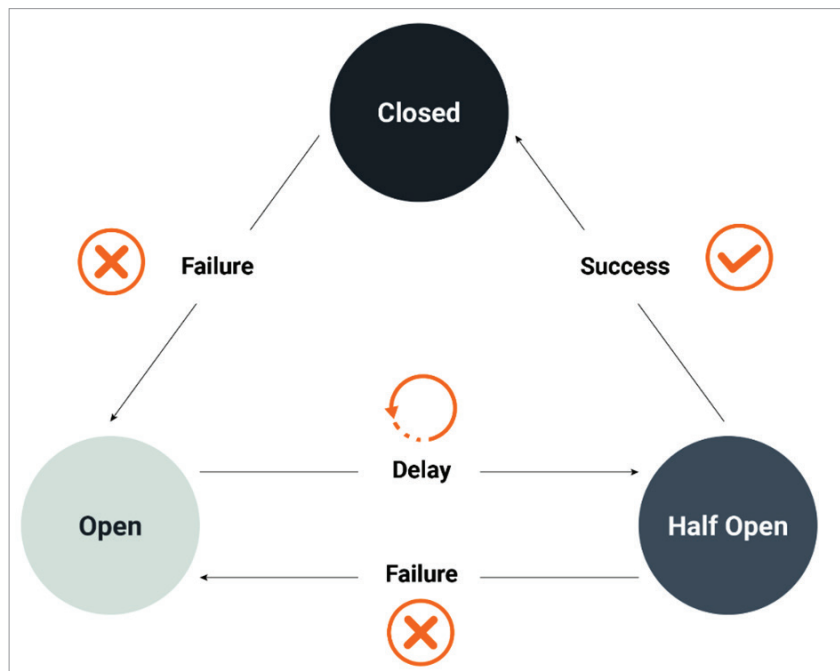


Figure 8: Methodology of circuit breaker pattern

```
Listing 1: Circuit breaker with RestTemplate
@Service
public class CompanyServiceClient {
    @Autowired
    private RestTemplate restTemplate;
    private static final String BASE_URL =
        "http://COMPANY-SERVICE/companies";
    @CircuitBreaker(name =
        "companyService", fallbackMethod =
        "fallbackGetCompanyById")
    public CompanyDTO getCompanyById(Long
        companyId)
    {
        return restTemplate.getForObject(BASE_
            URL + "/" + companyId, CompanyDTO.
            class);
    }
    // Fallback method called when the
    circuit is open
    public CompanyDTO
        fallbackGetCompanyById(Long companyId,
            Throwable throwable) {
        CompanyDTO fallbackCompany = new
            CompanyDTO ();
        fallbackCompany.setId(companyId);
        fallbackCompany.setName("Company
            information
            not available at the moment");
        return fallbackCompany;
    }
}
```

```
return fallbackCompany;
}
}
```

RabbitMQ integration

RabbitMQ is an open source message broker that facilitates communication between distributed applications or services by enabling them to exchange messages asynchronously. It implements the Advanced Message Queuing Protocol (AMQP) and is designed to ensure reliable, scalable, and fault-tolerant messaging. RabbitMQ, as outlined in Figure 9, allows producers to send messages to a queue, from which consumers can retrieve and process them. This decouples the components of an application, enabling them to communicate without direct dependencies.

RabbitMQ supports features such as message persistence (ensuring messages are not lost), message routing, load balancing, and delivery guarantees, which make it ideal for systems that require high throughput and fault tolerance. It is widely used in microservices architectures, event-driven systems, and for integrating